OGC2026 Technical Report

Abstract. The challenge asks for a schedule and a physical layout at the same time: every block needs a bay, an entry time, and a pose that a vertically descending crane can actually reach. Our starting observation is that the objective $w_1 Z_1 + w_2 Z_2 + w_3 Z_3$ reads only the pair (bay, entry time)—coordinates and orientation appear nowhere in it. Geometry is therefore a feasibility constraint that decides how much scheduling freedom exists, not a term to be optimised directly. Because the share of the objective carried by tardiness ranges from 0% to 97.4% across instances with nothing in between, we avoid density thresholds and instance classification entirely: a single portfolio of six operators competes for the time budget on measured payoff per second, and every hot loop lives in one of two C++ extensions. Our main contributions for the final round are an exact contact upper bound that skips 40.8% of the candidate cells in the construction while provably returning the identical solution, a 10.5× faster conflict-graph build verified by an unchanged edge count, and a position-independent geometric predicate that removed a floating-point inconsistency from the packing test. The most instructive result is negative: that same exact, verified 1.85× speedup made one instance 7.7% worse, because the operator schedule is driven by a wall clock and a faster search therefore takes a different path rather than a longer one.

1 Introduction

1.1 Challenge Problem & Main Difficulties

Three properties of this problem shaped everything we built.

The objective and the constraints read different variables. Tardiness, workload imbalance and preference are all functions of which bay a block occupies and when it enters. The block's x, y and orientation appear in none of them. Geometry enters only through feasibility: a layout that cannot accommodate a block pushes its entry later, and that is the only channel through which packing quality becomes objective value. Treating tightness as an objective term is a modelling error, and it is one we made early—weighting contact more heavily does not buy a better schedule, it buys a denser layout whose scheduling consequences are unmeasured.

The crane makes the packing three-dimensional. A block is lowered vertically, so a placement is legal only if every layer of the descending block clears everything already present along the whole descent. Two neighbours occupying the same floor cells but standing one and two layers high ration space completely differently. The scarce resource is height, not occupancy, and our contact measure is accumulated per layer for exactly this reason: a block standing beside a shorter one earns nothing for the layers that face open air.

Instances are not on a spectrum. Across the instances we measured, the tardiness share of the objective is 0% on the sparsest and 97.4% on the densest, with nothing in between; on the training set, Z_1 is exactly zero on a substantial fraction of instances. Where Z_1 vanishes the entry times are free and the problem is an assignment problem dominated by preference; where the yard is saturated the entry times are the whole objective. A hand-drawn density threshold has to guess where the switch lies, and the observed distribution gives no evidence about it. We therefore built a scheduler that measures which operator is paying rather than one that classifies the instance.

1.2 Contributions

We contribute a gate-free operator portfolio in which selection is by realised objective gain per second and slice sizing is a separate question answered by completion; an exact contact upper bound for the construction, verified per candidate cell rather than by comparing run outcomes; a 10.5× faster conflict-graph build whose acceptance test was an unchanged edge count rather than a stopwatch; and offset-relative geometry, which made the packing predicate position-independent and fixed a floating-point disagreement that had been misdiagnosed as a hashing problem. We also contribute a negative result we think is more useful than any of these, described in Section 2.3.

2 Algorithm Description

2.1 Overall Algorithm Framework

The entry point reserves $\min(0.2T, 40\,\mathrm{s})$ of the time limit for a final preference-repair pass and spends the rest on four independent worker processes. Each worker seeds a pool with a cheap always-feasible sequential schedule—the single guarantee that the algorithm never returns nothing, and the reason every other fallback path could be deleted—then runs one operator loop until its budget is exhausted. Workers differ in random seed and in their starting offset into six diversification axes. The best full objective across workers is taken, polished, and returned.

Feasibility is enforced inside the operators and re-checked outside them: the single acceptance function re-runs the provided feasibility checker on any solution that beats the incumbent and returns $+\infty$ if it fails. An operator may be wrong about feasibility, but a wrong answer cannot be accepted.

Six operators are available: a contact-beam construction; an evolutionary re-growth with crossover and path-relinking over (bay, dispatch-order) anchors, decoded by the same beam; a workload-balance repair; a preference repair; a CP-SAT bay reassignment; and the exact bay repacking of Section 3. Each receives one probe to establish a rate, after which the budget goes to whichever has the highest realised objective gain per second, with 15% exploration so a slow starter can recover.

Selection and sizing are different questions. Slice size is deliberately not driven by the selection signal. Sizing on payoff death-spirals: a construction that returns a perfectly good solution which merely fails to beat the incumbent scores zero gain, shrinks, and then can no longer finish at all—we measured fourteen calls returning four solutions while the objective moved from 48 840 to 56 841. Size is instead a completion question: a search operator that returned nothing was starved and is given more, one that returned anything fitted, and a repair pass is given what it actually used. The opening slices are asymmetric for the same reason, search operators taking a fifth of the budget and repair passes budget/$2n$; both symmetric alternatives were measured and both are worse.

2.2 Key Ideas that Succeeded

Contact beam with per-layer tightness. The construction is a beam search over dispatch order. Each state places the next block at the position minimising a weighted sum of negative contact, a positional sweep term and a preference penalty; survivors are completed by a rollout and ranked on the exact objective. Contact is accumulated per layer and normalised by the layer count, so a neighbour of matching height scores what a flat count would give and a mismatched one only a fraction. Summing layers without normalising inflates tightness by 1.35–1.95× against a fixed tardiness weight, and was measurably harmful.

An exact contact upper bound. Profiling showed 86% of scored cells were evaluated after the best cell of that scan had already been found. Our first attempt bounded the positional term and fired zero times in 112.9 million cells: the score is contact-dominated by more than an order of magnitude (a positional range of ≈ 2.3 against a contact range of ≈ 60), so a bound that knows only the sweep coordinate is never competitive.

Bounding the contact itself inverts this. Contact counts, for each occupied footprint cell, its four neighbours: one for a neighbour outside the bay, nothing for a neighbour inside the same footprint, and its weight for an occupied neighbour. Every occupied cell that can contribute therefore lies inside the footprint bounding box dilated by one and can be counted at most once per direction, giving

$$\mathrm{ct}(i_x, i_y) \;\le\; W(i_x, i_y) \;+\; 4\, w_{\max} \left| \mathcal{O} \cap B^{+1}(i_x, i_y) \right|, \tag{1}$$

where W counts cell–direction pairs pointing out of the bay, $\mathcal{O}$ is the occupied set and B^{+1} the dilated box. A two-dimensional occupancy prefix sum, built once per (bay, time window) at the same asymptotic cost as the occupancy map it accompanies, answers the rectangle in four reads; a cell whose best possible score cannot beat the incumbent is then skipped without any geometric test. The bound is applied only where it is proved, so the layered contact variant, which counts against per-layer occupancy that a flat prefix sum does not dominate, is excluded.

A conflict-graph build that watches its own clock. The repacking operator begins with an $O(n_{\mathrm{col}}^2)$ conflict graph that cannot be interrupted. Two columns can conflict only while both are present, so sorting by entry time and breaking the inner loop when the later column's entry passes the earlier one's exit skips 83% of the pairs without visiting them; the fields the filter reads were also moved into flat arrays. Neither change can alter which pairs conflict, so the edge count must come out identical, and that—not the clock—was the acceptance test. In production the same configuration now builds in 10.7–14.2 s where it previously cost about 90 s. The build additionally projects its own completion every 256 rows and abandons a configuration it cannot finish, stepping down to a cheaper one rather than losing the call.

Offset-relative geometry. Memoising the conflict predicate initially produced 36 extra edges. We assumed key aliasing; dumping the disagreements showed the predicate itself was position-dependent through floating-point rounding. Comparing origin outlines plus an offset makes the verdict independent of absolute position, which fixed the memo by fixing the predicate.

2.3 Key Ideas that Failed

A wall-clock schedule turns a speedup into a different answer. This is the result we would most like to pass on. Having verified that the contact upper bound is exact, that it returns bit-identical placements at fixed work, and that it is 1.85×, 1.66× and 1.09× faster on three instances of very different density, we enabled it and one instance got 7.7% worse.

The bound is not at fault, and neither is anything else in the search. The operator loop decides what to run next, and for how long, from elapsed seconds. A faster scan therefore fits more calls into the same budget, which shifts the diversification axis rotation, the contents of the solution pool, and the repacking operator's own call counter and hence its random seed. The repacking operator is handed a different incumbent, and on that instance one particular incumbent—worth 107 195 before repacking—is the only one from which the operator ever reaches 83 095, and 83 095 is in turn the only route to the best solution we have ever recorded there. Across the whole day, every run reaching that solution had 83 095 in its log and every run that did not, did not. With the bound off we reproduce it three times in four; with it on, never in two.

The lesson generalises beyond this bound. Making a time-budgeted portfolio faster does not make it search longer along the same path—it makes it take a different path, and there is no reason for the new one to be better. We priced it per instance, as the competition scores, and shipped with the bound disabled: −7.7% on one instance against +2.2% on another is a losing trade. The correct repair is to stop the schedule depending on a clock, which is Section 2.4.

A better incumbent can repack worse. The same investigation produced a second surprise. Handing the repacking operator solutions of increasing quality does not produce increasing improvements: incumbents worth 97 570, 100 685 and 107 195

yielded improvements of 9 580, 13 510 and 24 100 respectively. A tight solution has no slack left to reclaim, and the repacking operator eats slack. Its result is a function of the incumbent's layout, not of the incumbent's objective—yet the roster hands it the best-scoring solution every time.

Two optimisations that were simply wrong. A free-column bitset, intended to accelerate the packing search, was proved to produce identical results and then measured 1.6–1.9× *slower*: its saving scales with columns per block while its cost scales with edges, and we had assumed the wrong line was hot. Separately, we twice predicted the contact bound's benefit from its firing rate—1.3% of cells on one instance, 3.4% on another—and were wrong in the same direction both times (1.09× and 1.66× actual). Most cells are rejected by a cheap bitmap probe, whereas the cells a bound removes are precisely those that would have reached the exact geometric test. Cell counts are not seconds.

2.4 Further Improvement Plan

The first item is to remove the wall clock from the schedule's decisions. We localised where it enters: with the deadline lifted, the construction returns identical placements with and without the bound, but under the deadline it actually receives, the two return different solutions. The divergence therefore begins inside the construction's stopping rule, not in the operator loop, and the repair belongs there—the packing solver already takes an iteration cap for precisely this reason, so the pattern is established in our own code. Once the schedule depends on work done rather than seconds elapsed, the contact bound can be re-enabled and contribute its speed instead of a different trajectory.

Second, the repacking operator should sample the solution pool rather than always taking its best member, since its yield depends on slack rather than on quality; the neighbouring re-growth operator already samples the pool, so the mechanism exists.

Third, on the sparsest instances the reported objective is the minimum over a handful of independent repacking attempts. Under such a min-of-N objective, halving each attempt to double N is profitable whenever per-attempt quality degrades more slowly than the extra draw helps, and the 10.5× faster build is currently spent as a longer search inside the same number of attempts rather than as more of them.

3 Implementation Details

Python 3.12 orchestrates; two C++17 extension modules built with `pybind11` and OpenMP hold every hot loop. CP-SAT is used for one operator and the algorithm degrades to the remaining five if it is unavailable. Python holds only control flow—the scheduler, the operator definitions and the acceptance test—while every geometric predicate, every candidate scan and both search engines are C++. The construction engine parallelises over beam states with OpenMP and the portfolio parallelises over worker processes; the two are never nested.

Table 1. Training instances, 60 s each. n blocks, m bays.

Inst.	n	m	Z_1	Z_2	Z_3	Objective
1	100	2	0	157	2	1 499
2	100	3	0	144	15	3 690
3	100	3	0	1 542	186	43 320
4	100	2	0	2 278	0	15 946
5	150	3	0	1 654	228	45 778
6	150	3	0	2 346	173	42 372
7	150	3	0	2 335	239	52 195

The repacking operator. This operator lifts every block out of the most contested bay, adds the outside blocks that might want to enter it, and hands the whole set to an exact maximum-weight set-packing solver over space–time columns, weighted directly in objective units. A call has two phases of completely different character: an uninterruptible conflict-graph build, and a deadline-honouring local search. The solver therefore accepts a total-time bound and enforces it against its own measured build cost rather than a predicted one—an earlier version subtracted a predicted build from the search budget as well, double-charging it. The configuration ladder is expressed as fractions of each instance's own entry-time ceiling, so window widths are derived from the instance rather than fitted to it.

Verification instrumentation. The engines carry counters for cells visited, exact tests, cheap rejections, cells scored after the best was found, and cells skipped by the bound, plus a dedicated soundness mode that scores every cell the bound wanted to skip and counts those that would have won. On three instances of very different density that count was zero out of 45.9, 11.6 and 9.3 million skipped cells. This mattered: an earlier bound had passed an identity check that was meaningless because the branch never executed, so we stopped accepting outcome comparisons as evidence and began testing the property itself.

4 Computational Results

Table 1 reports all forty training instances at a uniform 60 s budget, decomposed into the three objective components. A uniform budget is the only comparable one—hidden limits vary per instance and are not disclosed—and all solutions pass the provided feasibility checker.

Two cautions belong with these numbers rather than after them. Run-to-run spread on the densest instance is approximately 112 000 on unchanged code, so differences smaller than that are noise. On the sparsest instance the reported objective is a minimum over the repacking operator's independent attempts within a run, and we therefore quote the reproducible value—reached in three runs of four—rather than a single sample.

Table 2. Effect of the successive changes. Differences inside the stated run-to-run spreads are not claimed as improvements.

Configuration	Sparse instance	Dense instance
Portfolio without the repacking operator	96 990	—
+ exact bay repacking	80 795	29 651 637
+ faster build, derived configuration ladder	80 795	29 566 129
+ contact bound enabled (*not shipped*)	86 975	29 566 129

Table 3. Contact upper bound: one construction call, identical arguments, deadline lifted. The digest hashes every returned placement, so "identical" means the whole solution.

Instance	cells skipped	unsound skips	without (s)	with (s) result
Sparse	45.9 M (40.8%)	0	103.52	55.99 identical, 1.85×
Mid	11.6 M (3.4%)	0	27.43	16.55 identical, 1.66×
Dense	9.3 M (1.3%)	0	102.72	94.61 identical, 1.09×

Speed, measured at identical work. A fixed-time A/B cannot evaluate an accelerated search, because both arms consume the same budget and the faster arm therefore reaches a different solution legitimately. We instead replay a single construction call with identical arguments and no deadline, and compare a digest over every returned placement.

The conflict-graph build was accepted only after its edge count was confirmed unchanged on every configuration tested; the resulting speedup is 10.5× in isolation and appears in production as a build cost of 10.7–14.2 s where the same configuration previously cost about 90 s.

5 Team Members' Contributions

The submission rules require the report to be anonymous, so members are identified by role only.

Member A led the algorithmic work: problem analysis, the decision to treat geometry as a feasibility constraint rather than an objective term, the design of the measured-payoff operator portfolio, the contact upper bound and its proof, the conflict-graph and geometry optimisations, and the diagnosis of the wall-clock dependence reported in Section 2.3.

Member B owned the codebase and the experimental process: repository and version management, packaging and submission builds, the measurement harnesses and the discipline of running every comparison on an unloaded machine, execution and bookkeeping of the full training and validation sweeps, and verification that reported numbers matched the logs that produced them.

Both members contributed to result analysis and to this report.

6 AI Use Declaration

Generative AI was used substantially in this work, as an interactive coding and analysis assistant (Anthropic Claude, through its command-line coding interface) throughout development.

Concretely, it implemented and refactored the C++ extension modules and the Python scheduler from specifications settled in discussion; derived the contact upper bound of Section 2.2 together with the conditions under which it is valid; wrote the measurement harnesses, including the per-cell soundness check and the identical-work replay behind Table 3; executed experiments and tabulated results; diagnosed defects, among them the position-dependence of the geometric predicate and a double-charged time budget; and drafted this report.

Algorithmic direction, prioritisation between competing ideas, and every decision to accept or reject a change remained with the authors. Every number reported here was produced by executing the submitted code on the challenge instances; no result in this report was generated or estimated by a language model.